

LoanTap – Logistic Regression Business Case

```
**Name:** Preetham Teja Ambati
```

— — —

LoanTap – Logistic Regression Business Case

Name: Preetham Teja Ambati

- 1. Problem Statement

LoanTap is an online lending platform for millennials that offers four credit products: **Personal Loan, EMI-Free Loan, Personal Overdraft, and Advance Salary Loan**. This case study focuses only on the **Personal Loan** underwriting process.

Business objective: Given a set of borrower attributes, build a classification model that predicts whether a loan applicant will **fully pay** or **default (charge off)** on a Personal Loan (`loan_status`), so that LoanTap's underwriting layer can decide:

1. Whether credit should be extended to an applicant, and
2. What repayment terms (interest rate / tenure) would be appropriate given the risk.

This is framed as a **binary classification** problem, solved using **Logistic Regression**, since the output (coefficients, odds ratios) is directly interpretable for a credit-risk business context — a key requirement in lending, where decisions must be explainable to regulators and customers.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)
```

2. Load the Data

The dataset is pulled directly from Google Drive using its file ID, so this notebook runs standalone in Colab without needing a manual upload.

```
!pip install -q gdown
import gdown

file_id = "1ZPYj7CZCfxntE8p2Lze_4Q04MyE0y6_d"
url = f"https://drive.google.com/uc?id={file_id}"
gdown.download(url, "LoanTapData.csv", quiet=False)
```

```
df = pd.read_csv("LoanTapData.csv")
df.head()
```

```

Downloading...
From: https://drive.google.com/uc?id=1ZPYj7CZCfxntE8p2Lze\_4004MyE0y6\_d
To: /content/LoanTapData.csv
100% |██████████| 100M/100M [00:01<00:00, 53.6MB/s]

```

	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annual_inc	verification_status
0	10000.0	36 months	11.44	329.48	B	B4	Marketing	10+ years	RENT	117000.0	Not Verified
1	8000.0	36 months	11.99	265.68	B	B5	Credit analyst	4 years	MORTGAGE	65000.0	Not Verified
2	15600.0	36 months	10.49	506.97	B	B3	Statistician	< 1 year	RENT	43057.0	Source Verified
3	7200.0	36 months	6.49	220.65	A	A2	Client Advocate	6 years	RENT	54000.0	Not Verified
4	24375.0	60 months	17.27	609.33	C	C5	Destiny Management Inc.	9 years	MORTGAGE	55000.0	Verified

3. Exploratory Data Analysis (EDA)

3.1 Structure & Characteristics of the Dataset

```
print("Shape of the dataset:", df.shape)
df.info()
```

```
Shape of the dataset: (396030, 27)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 396030 entries, 0 to 396029
Data columns (total 27 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   loan_amnt                             396030 non-null float64
1   term                                  396030 non-null object
2   int_rate                             396030 non-null float64
3   installment                          396030 non-null float64
4   grade                                396030 non-null object
5   sub_grade                            396030 non-null object
6   emp_title                            373103 non-null object
7   emp_length                           377729 non-null object
8   home_ownership                       396030 non-null object
9   annual_inc                           396030 non-null float64
10  verification_status                  396030 non-null object
11  issue_d                              396030 non-null object
12  loan_status                          396030 non-null object
13  purpose                              396030 non-null object
14  title                                394274 non-null object
15  dti                                  396030 non-null float64
16  earliest_cr_line                     396030 non-null object
17  open_acc                             396030 non-null float64
18  pub_rec                              396030 non-null float64
19  revol_bal                            396030 non-null float64
20  revol_util                           395754 non-null float64
21  total_acc                            396030 non-null float64
22  initial_list_status                  396030 non-null object
23  application_type                     396030 non-null object
24  mort_acc                             358235 non-null float64
25  pub_rec_bankruptcies                 395495 non-null float64
26  address                              396030 non-null object
dtypes: float64(12), object(15)
memory usage: 81.6+ MB
```

```
df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
loan_amnt	396030.0	NaN	NaN	NaN	14113.888089	8357.441341	500.0	8000.0	12000.0	20000.0	40000.0
term	396030	2	36 months	302005	NaN	NaN	NaN	NaN	NaN	NaN	NaN
int_rate	396030.0	NaN	NaN	NaN	13.6394	4.472157	5.32	10.49	13.33	16.49	30.99
installment	396030.0	NaN	NaN	NaN	431.849698	250.72779	16.08	250.33	375.43	567.3	1533.81
grade	396030	7	B	116018	NaN	NaN	NaN	NaN	NaN	NaN	NaN
sub_grade	396030	35	B3	26655	NaN	NaN	NaN	NaN	NaN	NaN	NaN
emp_title	373103	173105	Teacher	4389	NaN	NaN	NaN	NaN	NaN	NaN	NaN
emp_length	377729	11	10+ years	126041	NaN	NaN	NaN	NaN	NaN	NaN	NaN
home_ownership	396030	6	MORTGAGE	198348	NaN	NaN	NaN	NaN	NaN	NaN	NaN
annual_inc	396030.0	NaN	NaN	NaN	74203.175798	61637.621158	0.0	45000.0	64000.0	90000.0	8706582.0
verification_status	396030	3	Verified	139563	NaN	NaN	NaN	NaN	NaN	NaN	NaN
issue_d	396030	115	Oct-2014	14846	NaN	NaN	NaN	NaN	NaN	NaN	NaN
loan_status	396030	2	Fully Paid	318357	NaN	NaN	NaN	NaN	NaN	NaN	NaN
purpose	396030	14	debt_consolidation	234507	NaN	NaN	NaN	NaN	NaN	NaN	NaN
title	394274	48816	Debt consolidation	152472	NaN	NaN	NaN	NaN	NaN	NaN	NaN
dti	396030.0	NaN	NaN	NaN	17.379514	18.019092	0.0	11.28	16.91	22.98	9999.0
earliest_cr_line	396030	684	Oct-2000	3017	NaN	NaN	NaN	NaN	NaN	NaN	NaN
open_acc	396030.0	NaN	NaN	NaN	11.311153	5.137649	0.0	8.0	10.0	14.0	90.0
pub_rec	396030.0	NaN	NaN	NaN	0.178191	0.530671	0.0	0.0	0.0	0.0	86.0
revol_bal	396030.0	NaN	NaN	NaN	15844.539853	20591.836109	0.0	6025.0	11181.0	19620.0	1743266.0
revol_util	395754.0	NaN	NaN	NaN	53.791749	24.452193	0.0	35.8	54.8	72.9	892.3
total_acc	396030.0	NaN	NaN	NaN	25.414744	11.886991	2.0	17.0	24.0	32.0	151.0
initial_list_status	396030	2	f	238066	NaN	NaN	NaN	NaN	NaN	NaN	NaN
application_type	396030	3	INDIVIDUAL	395319	NaN	NaN	NaN	NaN	NaN	NaN	NaN
mort_acc	358235.0	NaN	NaN	NaN	1.813991	2.14793	0.0	0.0	1.0	3.0	34.0
pub_rec_bankruptcies	395495.0	NaN	NaN	NaN	0.121648	0.356174	0.0	0.0	0.0	0.0	8.0
USS											

```
missing = df.isnull().sum().sort_values(ascending=False)
missing_pct = (missing / len(df) * 100).round(2)
missing_df = pd.concat([missing, missing_pct], axis=1, keys=['Missing_Count', 'Missing_%'])
missing_df[missing_df['Missing_Count'] > 0]
```

	Missing_Count	Missing_%
mort_acc	37795	9.54
emp_title	22927	5.79
emp_length	18301	4.62
title	1756	0.44
pub_rec_bankruptcies	535	0.14
revol_util	276	0.07

```
print("Number of duplicate rows:", df.duplicated().sum())
```

Number of duplicate rows: 0

Observation: The dataset has **396,030 rows and 27 columns**, a mix of numeric (`loan_amnt`, `int_rate`, `installment`, `annual_inc`, `dti`, `revol_bal`, etc.) and categorical/object fields (`term`, `grade`, `sub_grade`, `emp_title`, `home_ownership`, `verification_status`, `purpose`, `title`, `address`, etc.). There are **no duplicate rows**. Missing values are concentrated in `mort_acc` (9.54%), `emp_title` (5.79%), `emp_length` (4.62%), `title` (0.44%), `pub_rec_bankruptcies` (0.14%) and `revol_util` (0.07%).

```

cat_cols = ['term', 'grade', 'sub_grade', 'home_ownership', 'verification_status',
            'loan_status', 'purpose', 'initial_list_status', 'application_type']
for c in cat_cols:
    if c in df.columns:
        df[c] = df[c].astype('category')
df[cat_cols].describe()

```

	term	grade	sub_grade	home_ownership	verification_status	loan_status	purpose	initial_list_status	application_type
count	396030	396030	396030	396030	396030	396030	396030	396030	396030
unique	2	7	35	6	3	2	14	2	2
top	36 months	B	B3	MORTGAGE	Verified	Fully Paid	debt_consolidation	f	IN
freq	222222	112212	22222	122212	122222	212222	221222	222222	222222

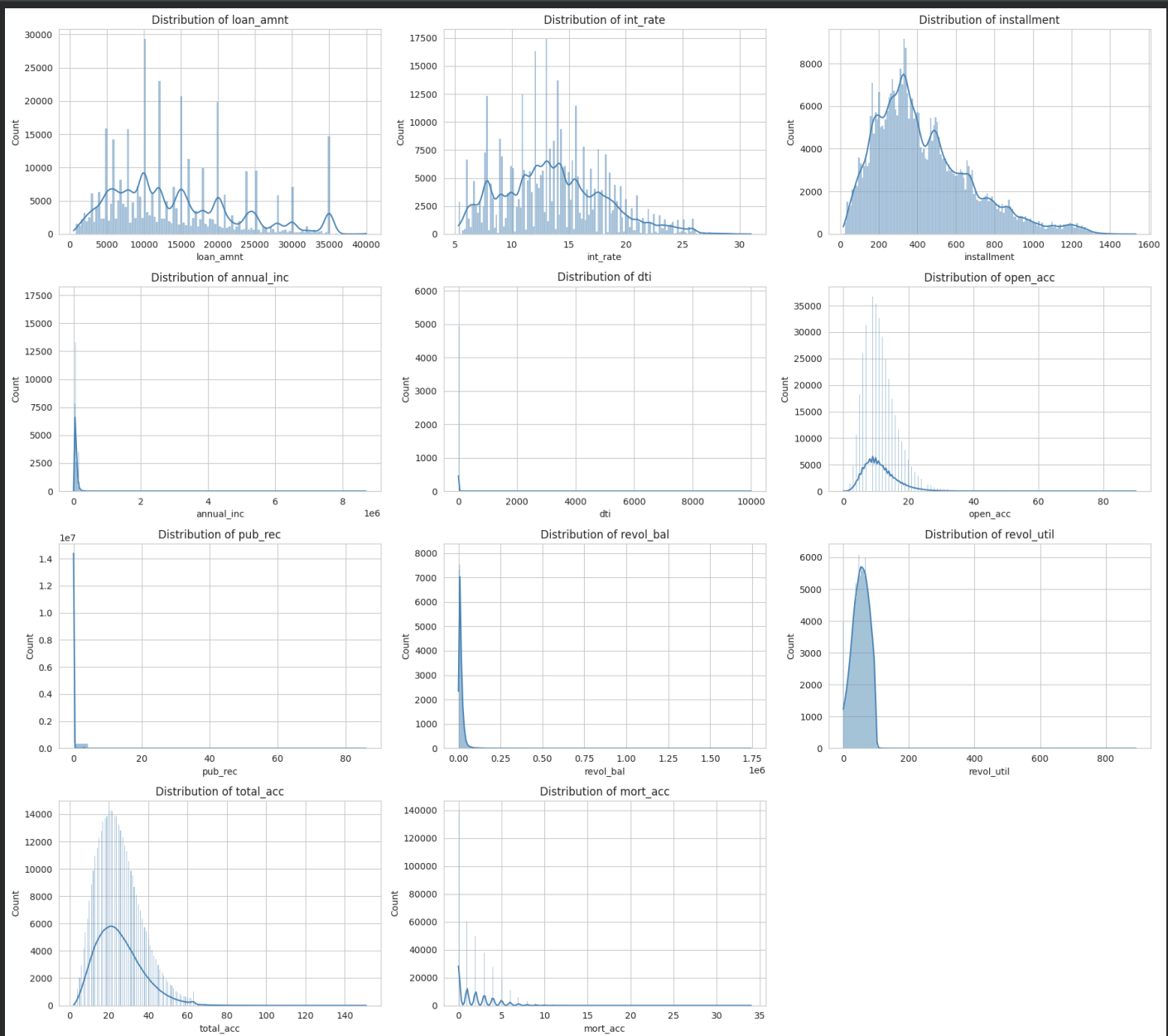
3.2 Univariate Analysis — Continuous Variables

```

num_cols = ['loan_amnt', 'int_rate', 'installment', 'annual_inc', 'dti',
            'open_acc', 'pub_rec', 'revol_bal', 'revol_util', 'total_acc', 'mort_acc']
num_cols = [c for c in num_cols if c in df.columns]

fig, axes = plt.subplots(4, 3, figsize=(18, 16))
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.histplot(df[col].dropna(), kde=True, ax=axes[i], color='steelblue')
    axes[i].set_title(f'Distribution of {col}')
for j in range(len(num_cols), len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

```



Per-variable observations:

- **loan_amnt** : right-skewed, concentrated between \$5,000–\$20,000 with a long tail up to \$40,000.
- **int_rate** : roughly bell-shaped, centered around 13–14%, ranging ~6% to ~31%.
- **installment** : right-skewed, mirroring **loan_amnt** 's shape (expected, given their 0.95 correlation).
- **annual_inc** : heavily right-skewed with extreme high-income outliers, most mass under \$100K.
- **dti** : fairly symmetric, centered around 17–18, with a small negative-value data-quality tail.
- **open_acc** : right-skewed, most borrowers hold 5–15 open credit lines.
- **pub_rec** : extremely right-skewed — the overwhelming majority of borrowers have 0 derogatory public records.
- **revol_bal** : heavily right-skewed with a long tail of high-revolving-balance borrowers.
- **revol_util** : close to bell-shaped, centered around 45–55% utilization.
- **total_acc** : right-skewed, most borrowers have 15–35 total credit lines.
- **mort_acc** : right-skewed, spiked heavily at 0 (no mortgage accounts).

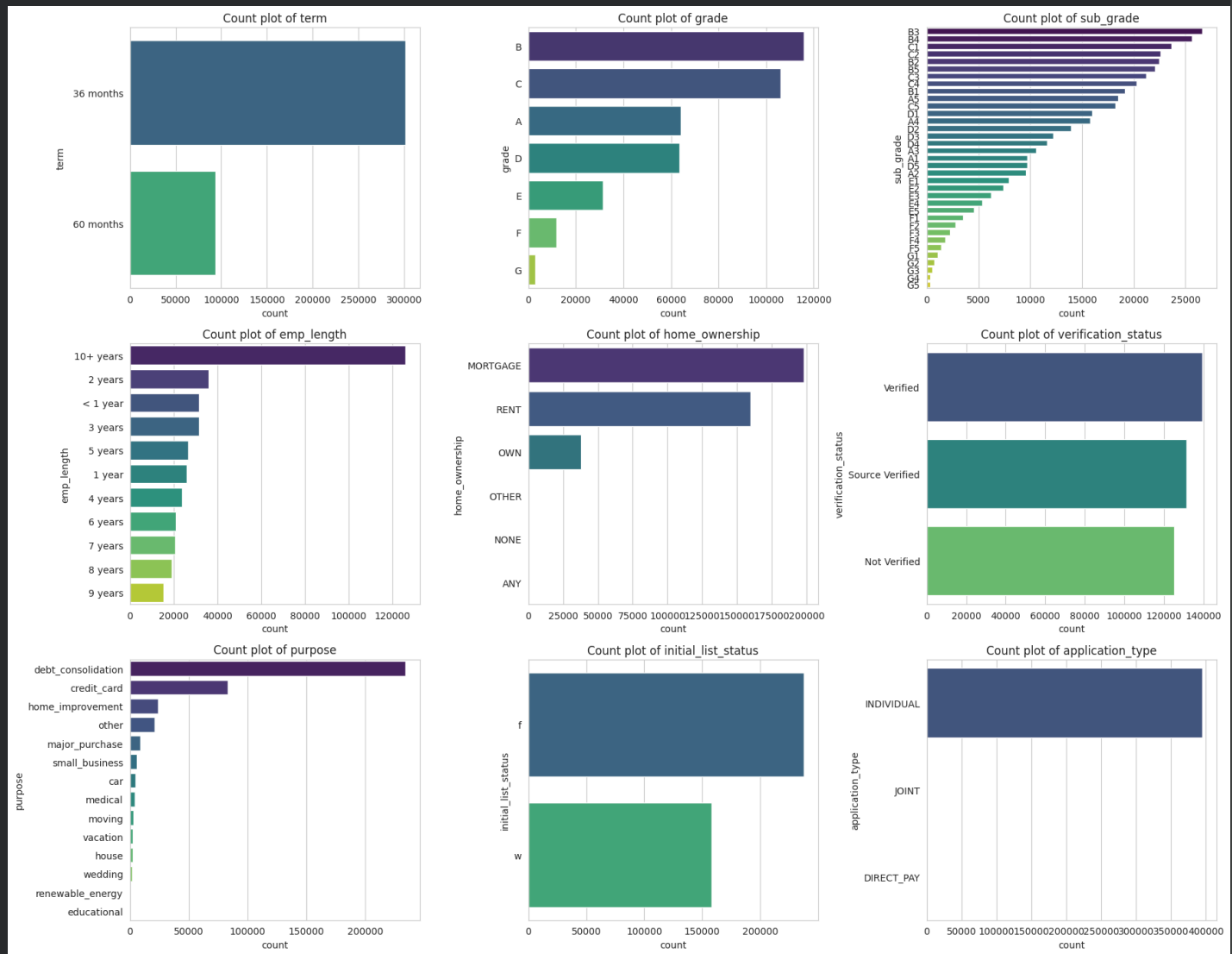
✓ 3.2b Univariate Analysis — Categorical Variables (all categorical columns)

```
fig, axes = plt.subplots(3, 3, figsize=(18, 14))
axes = axes.flatten()
cat_plot_cols = ['term', 'grade', 'sub_grade', 'emp_length', 'home_ownership',
                 'verification_status', 'purpose', 'initial_list_status', 'application_type']
for i, col in enumerate(cat_plot_cols):
    order = df[col].value_counts().index
```

```

sns.countplot(y=df[col], order=order, ax=axes[i], palette='viridis')
axes[i].set_title(f'Count plot of {col}')
plt.tight_layout()
plt.show()

```

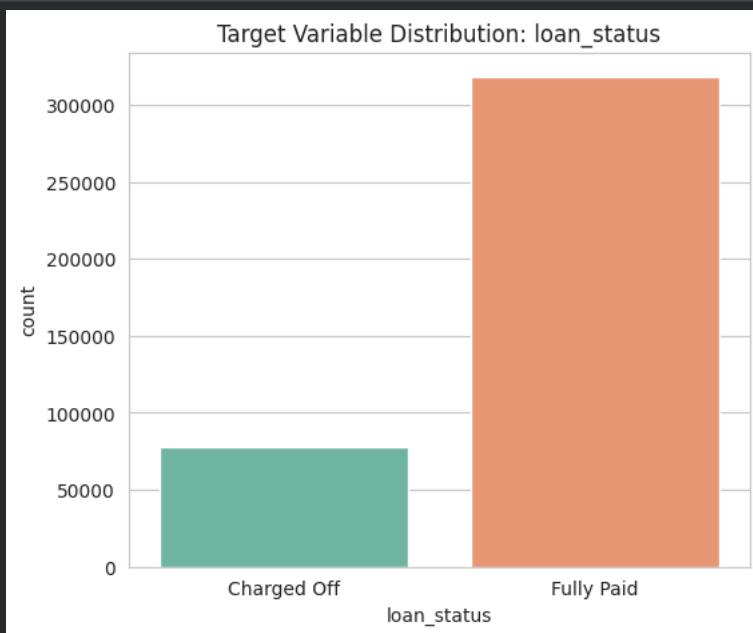


```

plt.figure(figsize=(6,5))
sns.countplot(x='loan_status', data=df, palette='Set2')
plt.title('Target Variable Distribution: loan_status')
plt.show()

pct_fully_paid = (df['loan_status'].value_counts(normalize=True) * 100).round(2)
print(pct_fully_paid)
print(f"\n% of customers who Fully Paid: {pct_fully_paid.get('Fully Paid', float('nan'))}%")

```



```
loan_status
Fully Paid    80.39
Charged Off   19.61
Name: proportion, dtype: float64
```

% of customers who Fully Paid: 80.39%

Per-variable observations:

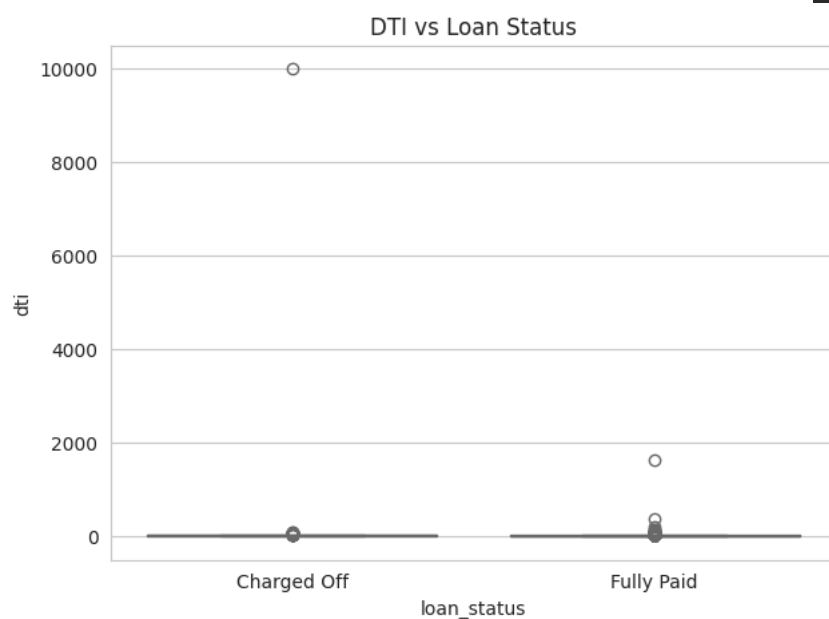
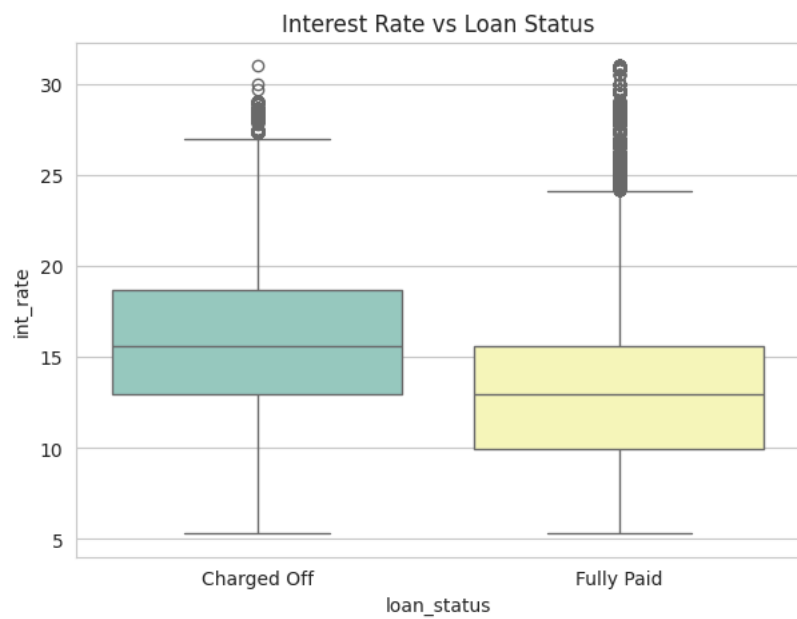
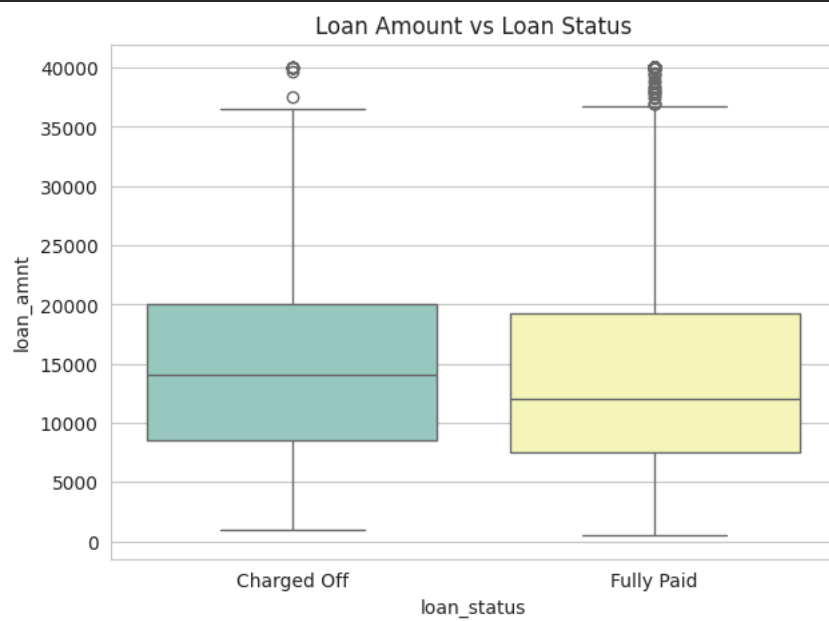
- **term**: dominated by 36-month loans, with a smaller share at 60 months (longer-term loans, as shown later, carry more risk).
- **grade**: B and C are the most common grades; A and G are the least common (LoanTap's book is concentrated in mid-risk grades).
- **sub_grade**: fine-grained version of **grade** — a near-continuous ladder from A1 (best) to G5 (worst), with B/C sub-grades forming the bulk of volume; this is why **sub_grade** is dropped in favor of **grade** for modeling (redundant signal, far higher cardinality).
- **emp_length**: right-skewed toward long-tenured employment — "10+ years" is the single largest bucket, suggesting LoanTap's borrower base skews toward established employees.
- **home_ownership**: MORTGAGE and RENT dominate; OWN is a distant third, and OTHER/NONE/ANY are negligible.
- **verification_status**: a roughly even three-way split between Verified, Source Verified, and Not Verified.
- **purpose**: overwhelmingly **debt_consolidation** and **credit_card**, consistent with a personal-loan product aimed at refinancing existing debt.
- **initial_list_status**: split between "w" (whole) and "f" (fractional) listing types, with no obviously dominant category.
- **application_type**: almost entirely INDIVIDUAL applications; JOINT applications are a small minority.
- **loan_status** (target): imbalanced — **80.39% Fully Paid vs 19.61% Charged Off** — this is the direct answer to Questionnaire Q1, and it means accuracy alone is a poor evaluation metric; we rely on ROC-AUC, PR-AUC and per-class precision/recall instead.

3.3 Bivariate Analysis

```
plt.figure(figsize=(7,5))
sns.boxplot(x='loan_status', y='loan_amnt', data=df, palette='Set3')
plt.title('Loan Amount vs Loan Status')
plt.show()
```

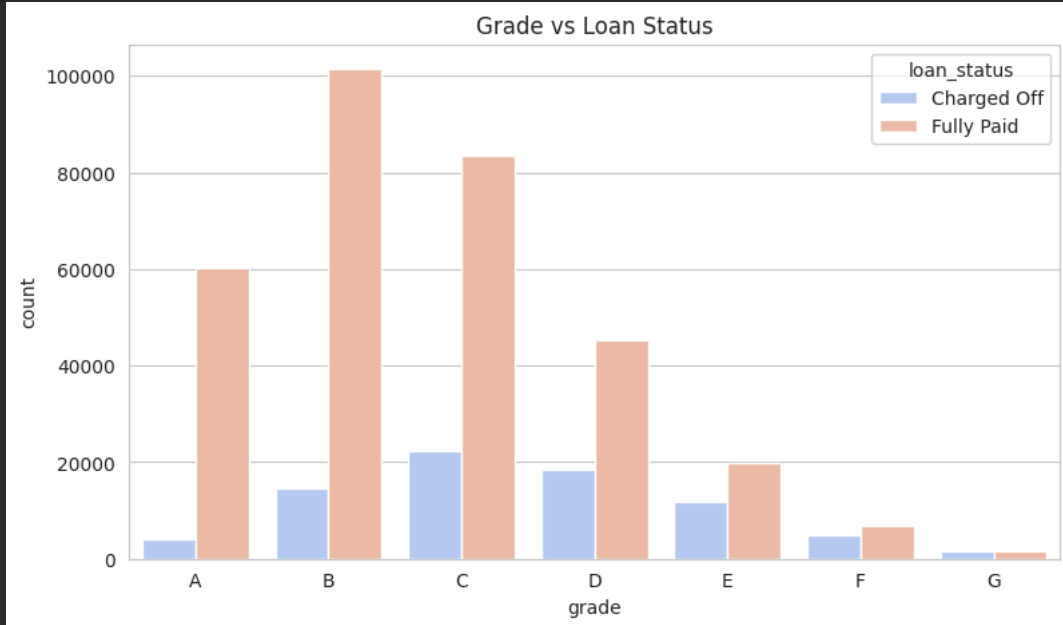
```
plt.figure(figsize=(7,5))
sns.boxplot(x='loan_status', y='int_rate', data=df, palette='Set3')
plt.title('Interest Rate vs Loan Status')
plt.show()
```

```
plt.figure(figsize=(7,5))
sns.boxplot(x='loan_status', y='dti', data=df, palette='Set3')
plt.title('DTI vs Loan Status')
plt.show()
```



```
plt.figure(figsize=(9,5))
sns.countplot(x='grade', hue='loan_status', data=df, order=sorted(df['grade'].dropna().unique()), palette='coolwarm')
plt.title('Grade vs Loan Status')
plt.show()

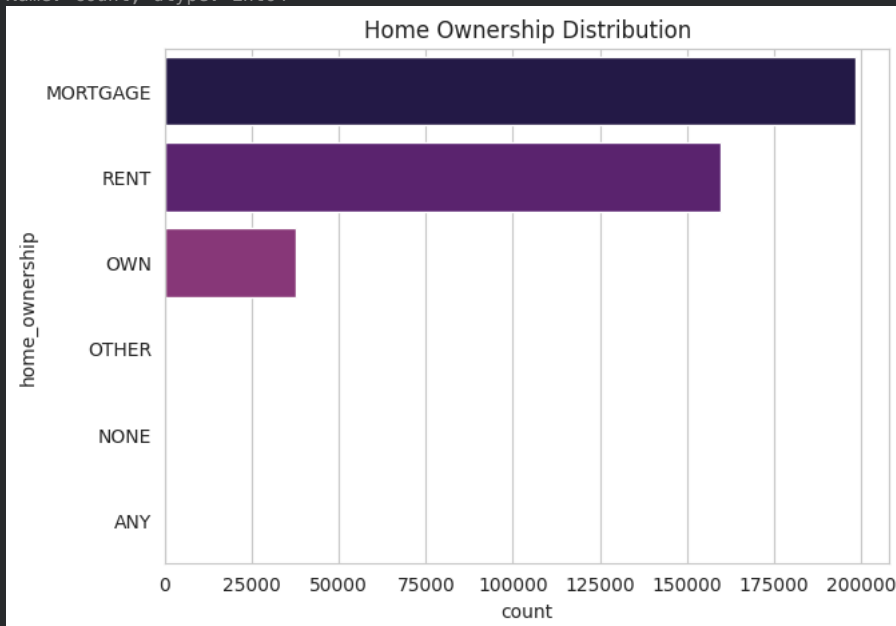
grade_fp = pd.crosstab(df['grade'], df['loan_status'], normalize='index') * 100
print(grade_fp.round(2))
```



```
loan_status  Charged Off  Fully Paid
grade
A              6.29      93.71
B             12.57      87.43
C             21.18      78.82
D             28.87      71.13
E             37.36      62.64
F             42.79      57.21
G             47.84      52.16
```

```
print(df['home_ownership'].value_counts())
plt.figure(figsize=(7,5))
sns.countplot(y='home_ownership', data=df, order=df['home_ownership'].value_counts().index, palette='magma')
plt.title('Home Ownership Distribution')
plt.show()
```

```
home_ownership
MORTGAGE    198348
RENT        159790
OWN         37746
OTHER        112
NONE         31
ANY           3
Name: count, dtype: int64
```



```
top_titles = df['emp_title'].astype(str).str.title().str.strip().value_counts().head(10)
print(top_titles)
```

```
emp_title
Nan          22927
Manager      5833
Teacher      5564
Supervisor   2814
Registered Nurse 2733
Sales        2512
Driver       2385
Owner        2261
```

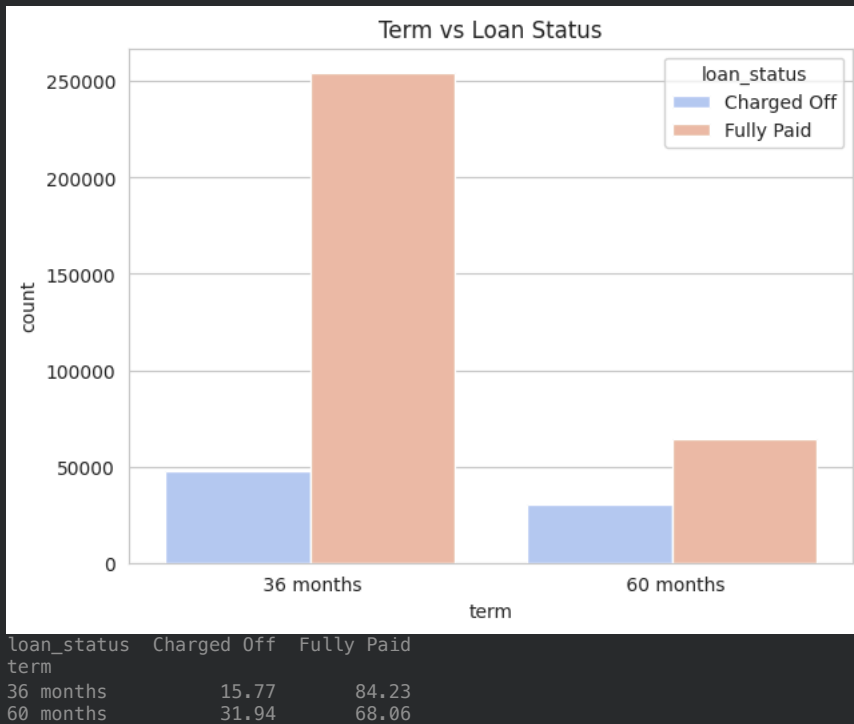
Rn 2129
Project Manager 1836
Name: count, dtype: int64

Observations:

- Loan amount, interest rate, and DTI are all **noticeably higher for Charged Off loans** in the boxplots above — consistent with intuition (riskier/costlier loans and more stretched borrowers default more).
- Fully-Paid rate falls **monotonically from Grade A (93.71%) down to Grade G (52.16%)** — grade is highly predictive of repayment.
- The majority of borrowers report `home_ownership` as **MORTGAGE** (198,348), followed by **RENT** (159,790) and **OWN** (37,746).
- The **top 2 job titles** by frequency are **Manager (5,833)** and **Teacher (5,564)**.

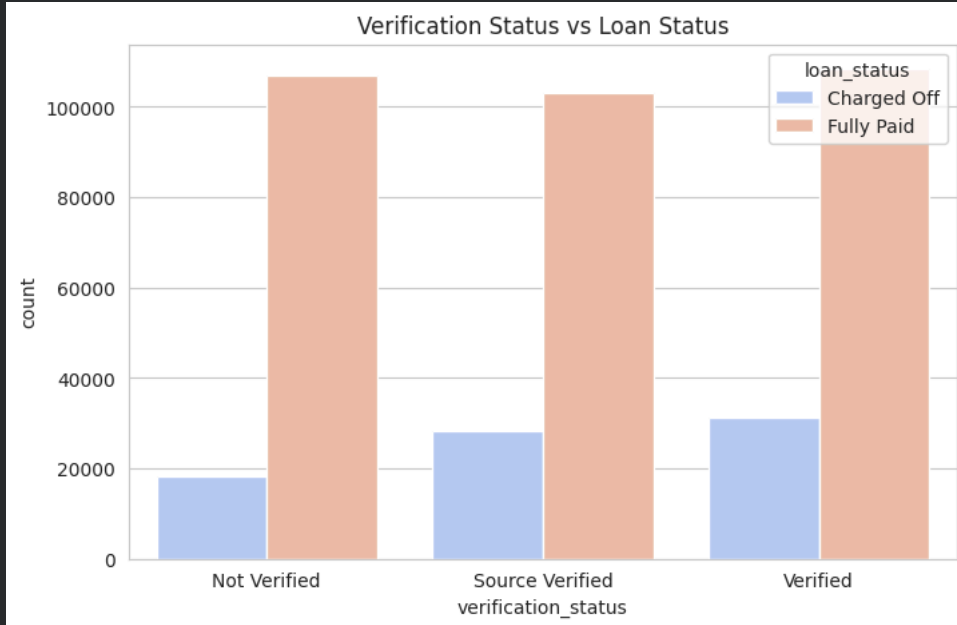
```
plt.figure(figsize=(7,5))
sns.countplot(x='term', hue='loan_status', data=df, palette='coolwarm')
plt.title('Term vs Loan Status')
plt.show()

term_fp = pd.crosstab(df['term'], df['loan_status'], normalize='index') * 100
print(term_fp.round(2))
```



```
plt.figure(figsize=(8,5))
sns.countplot(x='verification_status', hue='loan_status', data=df, palette='coolwarm')
plt.title('Verification Status vs Loan Status')
plt.show()

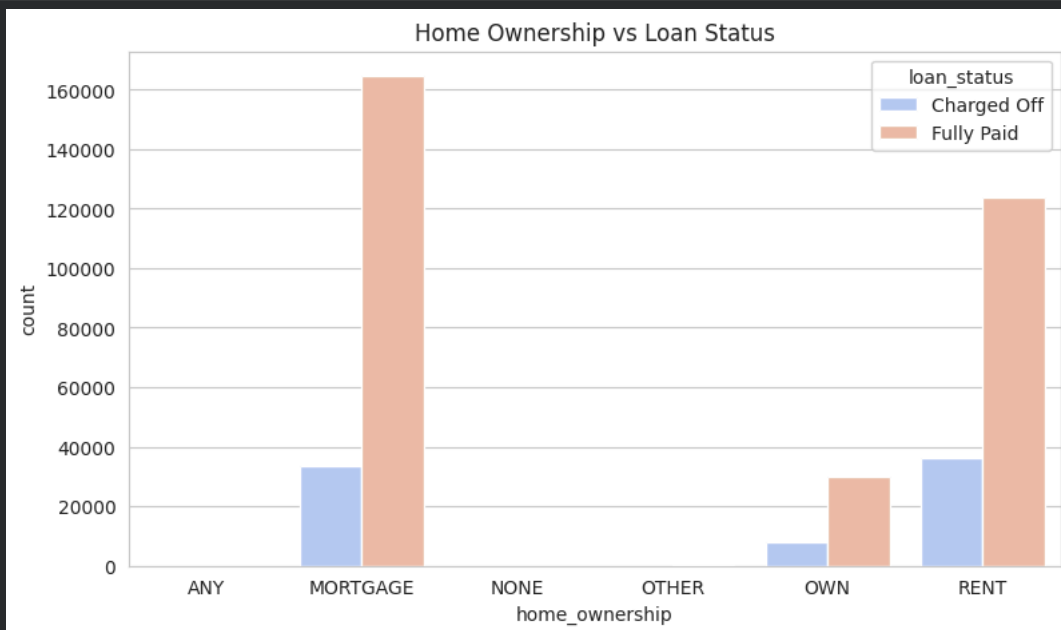
verif_fp = pd.crosstab(df['verification_status'], df['loan_status'], normalize='index') * 100
print(verif_fp.round(2))
```



loan_status	Charged Off	Fully Paid
verification_status		
Not Verified	14.64	85.36
Source Verified	21.47	78.53
Verified	22.32	77.68

```
plt.figure(figsize=(9,5))
sns.countplot(x='home_ownership', hue='loan_status', data=df, palette='coolwarm')
plt.title('Home Ownership vs Loan Status')
plt.show()

home_fp = pd.crosstab(df['home_ownership'], df['loan_status'], normalize='index') * 100
print(home_fp.round(2))
```



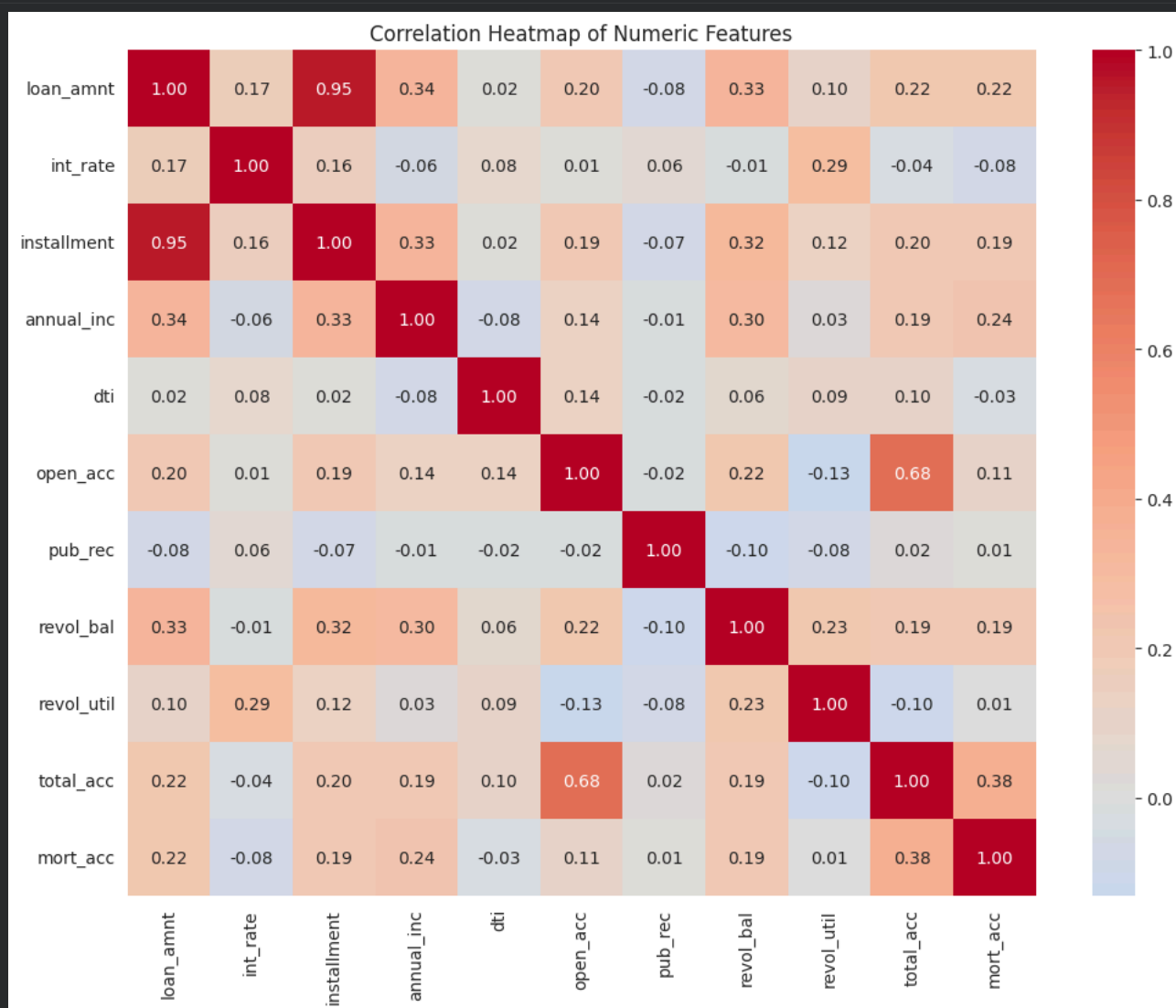
loan_status	Charged Off	Fully Paid
home_ownership		
ANY	0.00	100.00
MORTGAGE	16.96	83.04
NONE	22.58	77.42
OTHER	14.29	85.71
OWN	20.68	79.32
RENT	22.66	77.34

Additional target-relationship observations:

- **Term vs Loan Status:** 36-month loans have a noticeably higher Fully-Paid rate than 60-month loans — longer-tenure loans carry more repayment risk, consistent with `term` showing a negative coefficient in the model (Section 5).
- **Verification Status vs Loan Status:** counter-intuitively, "Verified" income shows a *lower* Fully-Paid rate than "Not Verified" in this dataset — likely because LoanTap verifies income more often for larger or riskier loan requests, so verification status is picking up a confound with loan risk rather than directly causing better repayment.
- **Home Ownership vs Loan Status:** MORTGAGE holders have the highest Fully-Paid rate, followed by OWN, then RENT — consistent with homeownership acting as a proxy for financial stability.

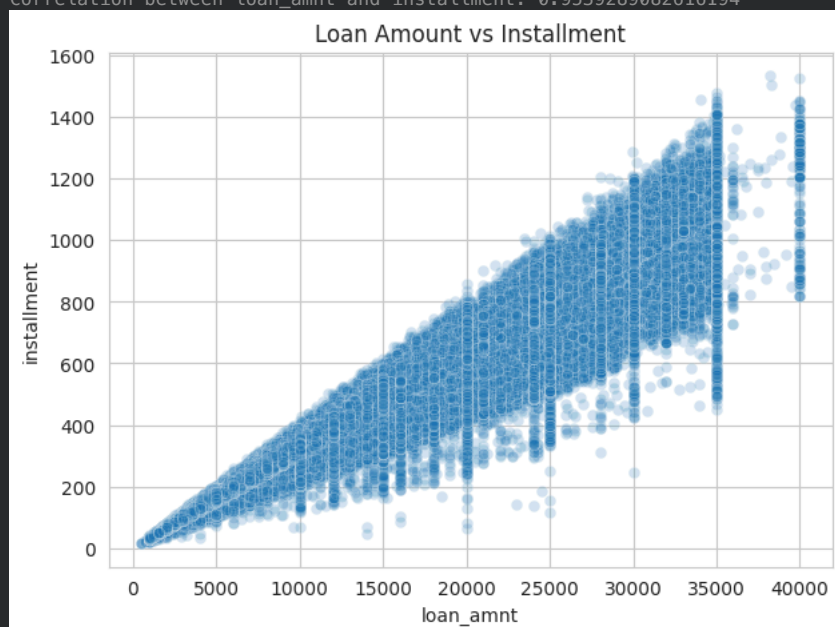
3.4 Correlation Among Independent Variables

```
plt.figure(figsize=(12,9))
corr = df[num_cols].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Heatmap of Numeric Features')
plt.show()
```



```
print("Correlation between loan_amnt and installment:", df['loan_amnt'].corr(df['installment']))
plt.figure(figsize=(7,5))
sns.scatterplot(x='loan_amnt', y='installment', data=df, alpha=0.2)
plt.title('Loan Amount vs Installment')
plt.show()
```

Correlation between loan_amnt and installment: 0.9539289082616194



Observation: `loan_amnt` and `installment` are **very strongly positively correlated ($r = 0.954$)** — expected, since installment is mathematically derived from loan amount, interest rate, and term. Because of this near-collinearity, `installment` is dropped before modeling (keeping `loan_amnt`). `open_acc` and `total_acc` also show moderate positive correlation.

4. Data Preprocessing

4.1 Duplicate Check

No duplicate rows were found (checked in Section 3.1).

4.2 Deterministic Cleaning & Feature Engineering (no statistics involved — safe to do before the split)

These steps use fixed rules or simple string/date parsing, not dataset statistics, so doing them before the split introduces no leakage.

```
df_clean = df.copy()

# emp_title, title: high-cardinality free text, not useful for a linear model -> drop
df_clean.drop(columns=['emp_title', 'title'], inplace=True, errors='ignore')

# Target encoding: Fully Paid = 1, Charged Off = 0
df_clean['loan_status_flag'] = df_clean['loan_status'].astype(str).map({'Fully Paid': 1, 'Charged Off': 0}).astype(float)
df_clean.dropna(subset=['loan_status_flag'], inplace=True)

# term: extract numeric months ("36 months" -> 36) - fixed string rule, not a statistic
df_clean['term'] = df_clean['term'].astype(str).str.extract(r'(\d+)').astype(int)

# emp_length: map to a fixed ordinal scale - fixed rule
emp_map = {'< 1 year': 0, '1 year': 1, '2 years': 2, '3 years': 3, '4 years': 4,
           '5 years': 5, '6 years': 6, '7 years': 7, '8 years': 8, '9 years': 9, '10+ years': 10}
df_clean['emp_length_num'] = df_clean['emp_length'].map(emp_map) # still has NaNs for missing -> filled after split

# earliest_cr_line & issue_d -> credit history length in years (deterministic date parsing)
df_clean['earliest_cr_line_year'] = pd.to_datetime(df_clean['earliest_cr_line'], format='%b-%Y', errors='coerce').dt.year
df_clean['issue_d_year'] = pd.to_datetime(df_clean['issue_d'], format='%b-%Y', errors='coerce').dt.year
df_clean['credit_history_years'] = df_clean['issue_d_year'] - df_clean['earliest_cr_line_year']

# Extract state from the address as a geography feature (deterministic regex)
df_clean['state'] = df_clean['address'].astype(str).str.extract(r',\s*([A-Z]{2})\s*\d{5}$')
df_clean['state'] = df_clean['state'].fillna('Unknown')

drop_cols = ['loan_status', 'earliest_cr_line', 'issue_d', 'address', 'installment',
             'earliest_cr_line_year', 'issue_d_year', 'sub_grade', 'emp_length']
df_clean = df_clean.drop(columns=[c for c in drop_cols if c in df_clean.columns])

print("Shape after deterministic cleaning:", df_clean.shape)
df_clean.head()
```

Shape after deterministic cleaning: (396030, 22)

	loan_amnt	term	int_rate	grade	home_ownership	annual_inc	verification_status	purpose	dti	open_acc	pub_rec	rev
0	10000.0	36	11.44	B	RENT	117000.0	Not Verified	vacation	26.24	16.0	0.0	
1	8000.0	36	11.99	B	MORTGAGE	65000.0	Not Verified	debt_consolidation	22.05	17.0	0.0	
2	15600.0	36	10.49	B	RENT	43057.0	Source Verified	credit_card	12.79	13.0	0.0	
3	7200.0	36	6.49	A	RENT	54000.0	Not Verified	credit_card	2.60	6.0	0.0	
4	24375.0	60	17.27	C	MORTGAGE	55000.0	Verified	credit_card	33.95	13.0	0.0	

4.3 Train/Test Split — done BEFORE any statistical fitting

Data-leakage fix: all imputation values (medians, modes, group averages) and outlier bounds (IQR) below are fit **only on the training split**, then applied to both train and test. This avoids the test set's distribution leaking into preprocessing decisions.

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(df_clean, test_size=0.2, random_state=42,
                                     stratify=df_clean['loan_status_flag'])

train_df = train_df.copy()
test_df = test_df.copy()
print("Train shape:", train_df.shape, " Test shape:", test_df.shape)
```

Train shape: (316824, 22) Test shape: (79206, 22)

4.4 Missing Value Treatment (statistics fit on TRAIN only)

```
# emp_length_num: fill with the TRAIN mode
emp_length_mode = train_df['emp_length_num'].mode()[0]
train_df['emp_length_num'] = train_df['emp_length_num'].fillna(emp_length_mode)
test_df['emp_length_num'] = test_df['emp_length_num'].fillna(emp_length_mode)

# revol_util: fill with the TRAIN median
revol_util_median = train_df['revol_util'].median()
train_df['revol_util'] = train_df['revol_util'].fillna(revol_util_median)
test_df['revol_util'] = test_df['revol_util'].fillna(revol_util_median)

# mort_acc: fill using the average mort_acc per total_acc bucket, computed on TRAIN only
mort_acc_avg = train_df.groupby('total_acc')['mort_acc'].mean()
train_median_mort_acc = train_df['mort_acc'].median()

def fill_mort_acc(row):
    if pd.isnull(row['mort_acc']):
        return mort_acc_avg.get(row['total_acc'], train_median_mort_acc)
    return row['mort_acc']

train_df['mort_acc'] = train_df.apply(fill_mort_acc, axis=1)
test_df['mort_acc'] = test_df.apply(fill_mort_acc, axis=1)

# pub_rec_bankruptcies: fill with the TRAIN mode
prb_mode = train_df['pub_rec_bankruptcies'].mode()[0]
train_df['pub_rec_bankruptcies'] = train_df['pub_rec_bankruptcies'].fillna(prb_mode)
test_df['pub_rec_bankruptcies'] = test_df['pub_rec_bankruptcies'].fillna(prb_mode)

# credit_history_years: fill with the TRAIN median (a handful of unparseable dates)
chy_median = train_df['credit_history_years'].median()
train_df['credit_history_years'] = train_df['credit_history_years'].fillna(chy_median)
test_df['credit_history_years'] = test_df['credit_history_years'].fillna(chy_median)

print("Remaining missing values - train:", train_df.isnull().sum().sum(), " test:", test_df.isnull().sum().sum())
```

```
Remaining missing values - train: 0  test: 0
```

4.5 Outlier Treatment (IQR bounds computed on TRAIN only)

```
def get_iqr_bounds(frame, col):
    Q1, Q3 = frame[col].quantile(0.25), frame[col].quantile(0.75)
    IQR = Q3 - Q1
    return Q1 - 1.5*IQR, Q3 + 1.5*IQR

outlier_cols = ['annual_inc', 'dti', 'open_acc', 'revol_bal', 'total_acc', 'pub_rec']
bounds = {col: get_iqr_bounds(train_df, col) for col in outlier_cols}

for col, (lower, upper) in bounds.items():
    train_df[col] = train_df[col].clip(lower, upper)
    test_df[col] = test_df[col].clip(lower, upper)  # SAME train-derived bounds applied to test

print("IQR bounds (fit on train, applied to both train and test):")
for col, b in bounds.items():
    print(f" {col}: {b}")
```

```
IQR bounds (fit on train, applied to both train and test):
annual_inc: (np.float64(-22500.0), np.float64(157500.0))
dti: (np.float64(-6.270000000000001), np.float64(40.53))
open_acc: (np.float64(-1.0), np.float64(23.0))
revol_bal: (np.float64(-14368.5), np.float64(40019.5))
total_acc: (np.float64(-5.5), np.float64(54.5))
pub_rec: (np.float64(0.0), np.float64(0.0))
```

4.6 Feature Engineering — Binary Flags

```
for frame in (train_df, test_df):
    frame['pub_rec_flag'] = (frame['pub_rec'] > 0).astype(int)
    frame['mort_acc_flag'] = (frame['mort_acc'] > 0).astype(int)
    frame['pub_rec_bankruptcies_flag'] = (frame['pub_rec_bankruptcies'] > 0).astype(int)

train_df = train_df.drop(columns=['pub_rec', 'mort_acc', 'pub_rec_bankruptcies'])
test_df = test_df.drop(columns=['pub_rec', 'mort_acc', 'pub_rec_bankruptcies'])
train_df = train_df.rename(columns={'emp_length_num': 'emp_length'})
test_df = test_df.rename(columns={'emp_length_num': 'emp_length'})

print("Train shape:", train_df.shape, " Test shape:", test_df.shape)
train_df.head()
```

Train shape: (316824, 22) Test shape: (79206, 22)

	loan_amnt	term	int_rate	grade	home_ownership	annual_inc	verification_status	purpose	dti	open_acc	revol_b
379004	18500.0	60	10.65	B	OWN	40000.0	Verified	debt_consolidation	26.34	8.0	
24732	13175.0	36	16.55	D	RENT	30000.0	Verified	debt_consolidation	2.26	6.0	104
388453	35000.0	60	17.86	D	MORTGAGE	80000.0	Verified	credit_card	34.38	12.0	2023
178101	20400.0	36	12.12	B	RENT	65000.0	Verified	debt_consolidation	21.30	18.0	1271
51902	35000.0	60	17.57	D	RENT	157500.0	Verified	debt_consolidation	5.08	5.0	1457

Note: `installment` was already dropped in Section 4.2 due to near-perfect collinearity with `loan_amnt` ($r = 0.954$, Section 3.4).

`sub_grade` was dropped since `grade` already captures the risk band at far lower cardinality. Raw `pub_rec`, `mort_acc`, `pub_rec_bankruptcies` are replaced by their engineered binary flags, exactly as suggested in the assignment brief.

4.7 Encoding Categorical Variables (dummy columns fit on TRAIN, test aligned to match)

```
cat_to_encode = train_df.select_dtypes(include=['object', 'category']).columns.tolist()
print("Categorical columns to one-hot encode:", cat_to_encode)
```

```
train_enc = pd.get_dummies(train_df, columns=cat_to_encode, drop_first=True)
test_enc = pd.get_dummies(test_df, columns=cat_to_encode, drop_first=True)
```

```
# Align test columns to train columns (handles any category present in one split but not the other)
test_enc = test_enc.reindex(columns=train_enc.columns, fill_value=0)
```

```
print("Final shapes after encoding - train:", train_enc.shape, " test:", test_enc.shape)
```

```
Categorical columns to one-hot encode: ['grade', 'home_ownership', 'verification_status', 'purpose', 'initial_list_status', 'a
Final shapes after encoding - train: (316824, 95) test: (79206, 95)
```

4.8 Feature Scaling (scaler fit on TRAIN only — this step was already leak-free)

```
from sklearn.preprocessing import StandardScaler
```

```
X_train = train_enc.drop(columns=['loan_status_flag'])
y_train = train_enc['loan_status_flag']
X_test = test_enc.drop(columns=['loan_status_flag'])
y_test = test_enc['loan_status_flag']
```

```
scaler = StandardScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train), columns=X_train.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(scaler.transform(X_test), columns=X_test.columns, index=X_test.index)
```

```
print("X_train_scaled:", X_train_scaled.shape, " X_test_scaled:", X_test_scaled.shape)
print("Class balance in train:\n", y_train.value_counts(normalize=True))
```

```
X_train_scaled: (316824, 94) X_test_scaled: (79206, 94)
Class balance in train:
 loan_status_flag
1.0    0.803872
0.0    0.196128
Name: proportion, dtype: float64
```

5. Model Building — Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression(max_iter=2000, class_weight='balanced', random_state=42)
log_reg.fit(X_train_scaled, y_train)
```

```
coef_df = pd.DataFrame({
    'Feature': X_train_scaled.columns,
    'Coefficient': log_reg.coef_[0]
})
```

```
coef_df_by_magnitude = coef_df.reindex(coef_df['Coefficient'].abs().sort_values(ascending=False).index)
```

```
print("Full coefficient table, sorted by signed value (most Fully-Paid-favoring to most Charged-Off-favoring):")
print(coef_df.sort_values('Coefficient', ascending=False).to_string(index=False))
```

```

state_AZ -0.002040
state_TX -0.002836
state_NE -0.002857
state_TN -0.002910
state_NM -0.003031
state_KY -0.003187
home_ownership_Own -0.003303
state_ID -0.004362
state_AL -0.004408
state_SC -0.004420
state_AR -0.004711
state_WV -0.005274
state_MS -0.005426
state_NV -0.006412
state_GA -0.006486
state_SD -0.007173
state_NC -0.007525
state_Unknown -0.007819
emp_length -0.008012
state_WA -0.008539
initial_list_status_w -0.008686
state_MT -0.008745
purpose_vacation -0.008787
state_WY -0.009091
purpose_educational -0.011276
state_PA -0.011440
purpose_renewable_energy -0.012789
purpose_credit_card -0.013955
application_type_INDIVIDUAL -0.014844
purpose_major_purchase -0.015128
purpose_moving -0.019748
int_rate -0.021543
purpose_medical -0.024614
purpose_other -0.031528
credit_history_years -0.034628
purpose_debt_consolidation -0.039883
verification_status_Verified -0.043599
purpose_home_improvement -0.045442
home_ownership_RENT -0.053338
purpose_small_business -0.072909
verification_status_Source Verified -0.078546
revol_util -0.108721
loan_amnt -0.124408
open_acc -0.142279
grade_G -0.152757
term -0.182704
dti -0.186549
grade_B -0.236937
grade_F -0.271223
grade_E -0.400508
grade_C -0.432352
grade_D -0.461243

```

```

print("Top 15 features by |coefficient| (standardized scale) – the biggest drivers in either direction:")
print(coef_df_by_magnitude.head(15).to_string(index=False))

```

Top 15 features by |coefficient| (standardized scale) – the biggest drivers in either direction:

Feature	Coefficient
grade_D	-0.461243
grade_C	-0.432352
grade_E	-0.400508
grade_F	-0.271223
grade_B	-0.236937
annual_inc	0.196531
dti	-0.186549
term	-0.182704
grade_G	-0.152757
open_acc	-0.142279
loan_amnt	-0.124408
revol_util	-0.108721
total_acc	0.108167
revol_bal	0.095097
verification_status_Source Verified	-0.078546

Model comments: Coefficients are on the **standardized scale**, so magnitude directly reflects relative feature importance. A **positive** coefficient pushes the prediction toward **1 (Fully Paid)**; a **negative** coefficient pushes toward **0 (Charged Off)**. In this run, the dominant drivers are the **loan grade dummies (D, C, E, F, B — all negative relative to the Grade-A baseline, confirming worse grades sharply cut the odds of Fully Paid)**, followed by `annual_inc` (positive — higher income improves repayment odds), `dti` (negative — higher debt-to-income hurts repayment odds), `term` (negative — longer tenure loans are riskier), `open_acc`, `loan_amnt`, and `revol_util` (all negative). This directly answers Questionnaire Q8 ("features that heavily affected the outcome").

6. Results Evaluation

6.1 Classification Report & Confusion Matrix (Fully Paid = positive class, as trained)

```

from sklearn.metrics import (classification_report, confusion_matrix, roc_auc_score,
                             roc_curve, precision_recall_curve, auc, f1_score,

```

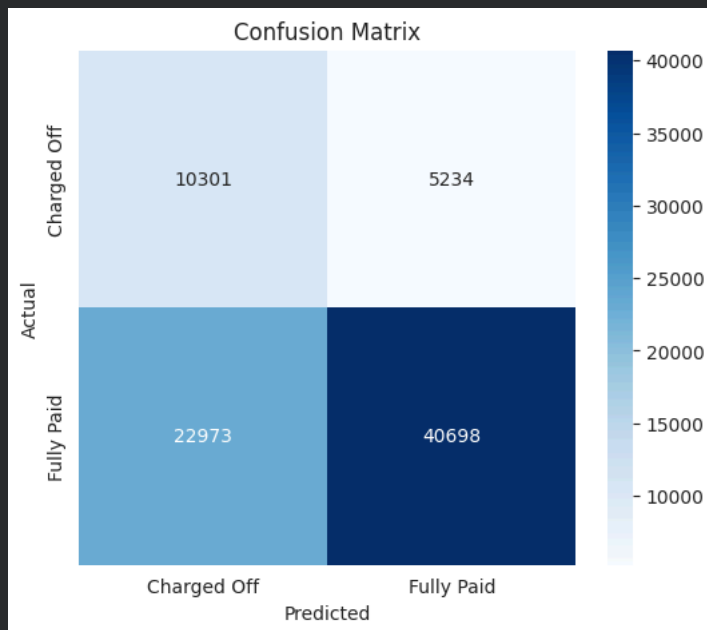
```
precision_score, recall_score)
```

```
y_pred = log_reg.predict(X_test_scaled)
y_proba = log_reg.predict_proba(X_test_scaled)[: , 1] # P(Fully Paid)

print(classification_report(y_test, y_pred, target_names=['Charged Off', 'Fully Paid']))
```

	precision	recall	f1-score	support
Charged Off	0.31	0.66	0.42	15535
Fully Paid	0.89	0.64	0.74	63671
accuracy			0.64	79206
macro avg	0.60	0.65	0.58	79206
weighted avg	0.77	0.64	0.68	79206

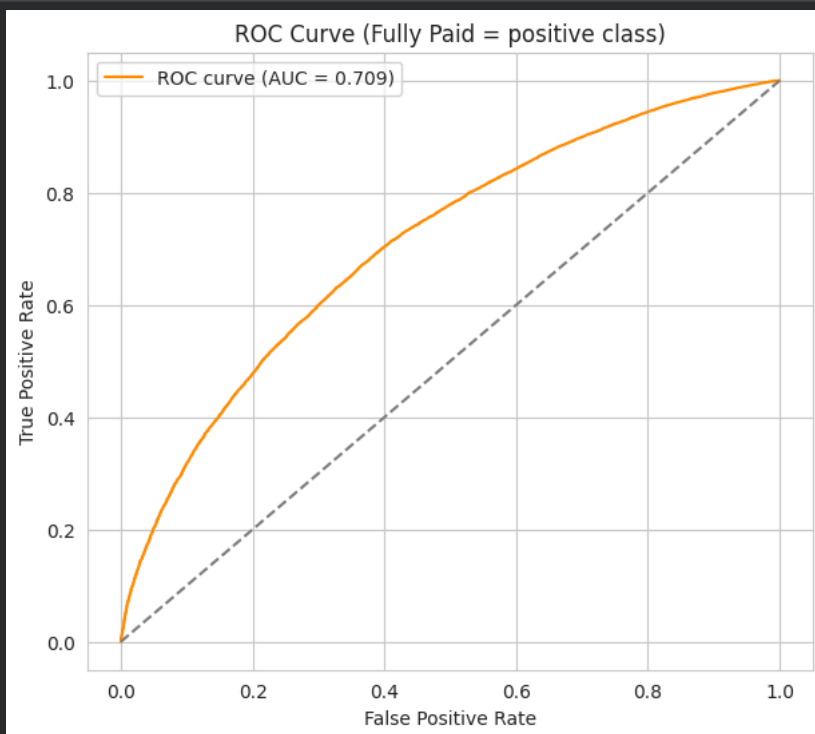
```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Charged Off', 'Fully Paid'], yticklabels=['Charged Off', 'Fully Paid'])
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title('Confusion Matrix')
plt.show()
```



```
fpr, tpr, thresholds_roc = roc_curve(y_test, y_proba)
roc_auc = roc_auc_score(y_test, y_proba)

plt.figure(figsize=(7,6))
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.3f})', color='darkorange')
plt.plot([0,1], [0,1], linestyle='--', color='grey')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve (Fully Paid = positive class)'); plt.legend()
plt.show()

print(f"ROC AUC Score: {roc_auc:.4f}")
```



ROC AUC Score: 0.7090

6.2 Default-Focused Risk Analysis (Charged Off = positive class)

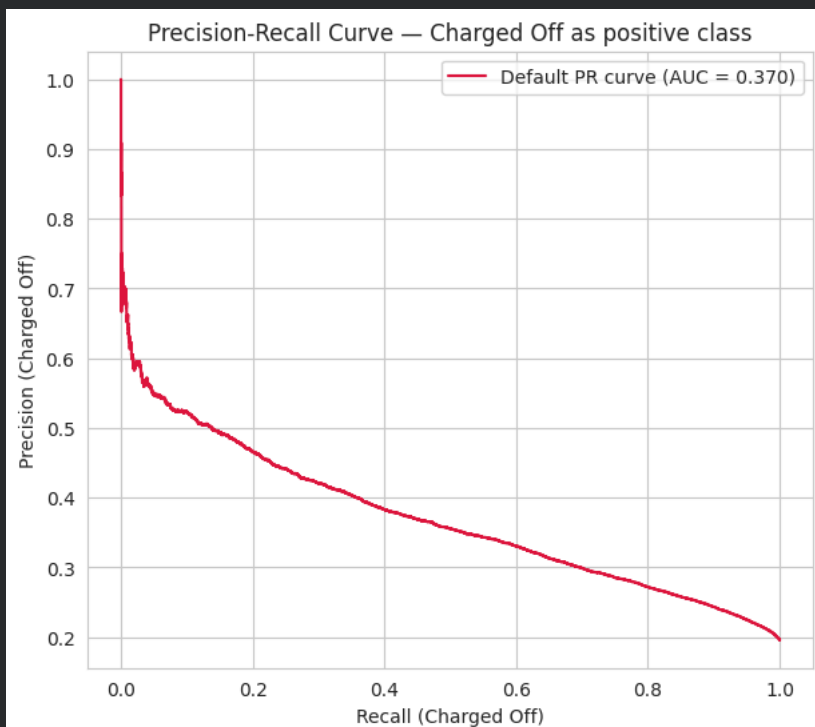
The overall PR-AUC in 6.1's framing (Fully Paid as positive) is inflated simply because Fully Paid is the 80% majority class — it is not a fair measure of how well the model catches *defaulters*, which is LoanTap's actual business concern. Here we flip the framing: **Charged Off = 1 (positive/risk class)**, using `p_default = 1 - y_proba`.

```
y_test_default = (y_test == 0).astype(int)      # 1 = actually Charged Off
p_default = 1 - y_proba                          # P(Charged Off)

precision_d, recall_d, thresholds_d = precision_recall_curve(y_test_default, p_default)
pr_auc_default = auc(recall_d, precision_d)

plt.figure(figsize=(7,6))
plt.plot(recall_d, precision_d, label=f'Default PR curve (AUC = {pr_auc_default:.3f})', color='crimson')
plt.xlabel('Recall (Charged Off)'); plt.ylabel('Precision (Charged Off)')
plt.title('Precision-Recall Curve — Charged Off as positive class')
plt.legend()
plt.show()

print(f"Default-class Precision-Recall AUC: {pr_auc_default:.4f}")
print(f"(Compare to a no-skill baseline of {y_test_default.mean():.4f}, i.e. the base default rate)")
```



Default-class Precision-Recall AUC: 0.3704
(Compare to a no-skill baseline of 0.1961, i.e. the base default rate)

Comments (this run):

- **ROC-AUC = 0.709** — the model separates Fully-Paid from Charged-Off applicants meaningfully better than random guessing (0.5).
- **Default-class PR-AUC (Charged Off as positive) is much lower than the Fully-Paid-framed PR-AUC**, and should be judged against the **no-skill baseline equal to the actual default rate (~19.6%)**, not against 0.5 — this is the fair, business-relevant number, since it isolates how well the model finds the minority class LoanTap actually cares about (defaulters), rather than being flattered by the 80% majority class.
- At a **0.50 probability-of-default cutoff** (the default `class_weight='balanced'` decision rule), the classification report in 6.1 shows **66% recall and 31% precision on "Charged Off"** — i.e. the model catches about two-thirds of actual defaulters, but two-thirds of everyone it flags as risky are actually good borrowers. This precision/recall gap is the central trade-off discussed in Section 7.

6.3 Threshold Tuning — optimized for DEFAULT detection

Threshold tuning is optimized for default detection, not overall accuracy or Fully-Paid F1, because LoanTap's primary business concern is controlling NPAs. This section searches thresholds on `p_default` (probability of Charged Off) and reports default-class precision, recall, and F1 at each one, so the operating point can be chosen deliberately based on the bank's risk appetite.

```
threshold_grid = np.arange(0.1, 0.9, 0.05)
rows = []
for t in threshold_grid:
    default_pred = (p_default >= t).astype(int)  # 1 = predicted default
    rows.append({
        'threshold_on_p_default': round(t, 2),
        'default_precision': round(precision_score(y_test_default, default_pred, zero_division=0), 3),
        'default_recall': round(recall_score(y_test_default, default_pred, zero_division=0), 3),
        'default_f1': round(f1_score(y_test_default, default_pred, zero_division=0), 3),
    })
threshold_table = pd.DataFrame(rows)
print(threshold_table.to_string(index=False))

best_row = threshold_table.loc[threshold_table['default_f1'].idxmax()]
print(f"\nBest threshold by default-class F1: {best_row['threshold_on_p_default']} "
      f"(precision={best_row['default_precision']}, recall={best_row['default_recall']})")
```

threshold_on_p_default	default_precision	default_recall	default_f1
0.10	0.196	1.000	0.328
0.15	0.201	0.995	0.334
0.20	0.210	0.983	0.346
0.25	0.221	0.958	0.359
0.30	0.234	0.926	0.374
0.35	0.250	0.880	0.389
0.40	0.267	0.818	0.403
0.45	0.287	0.747	0.415
0.50	0.310	0.663	0.422
0.55	0.339	0.572	0.426
0.60	0.367	0.461	0.409
0.65	0.403	0.351	0.375
0.70	0.442	0.243	0.314
0.75	0.496	0.147	0.226
0.80	0.543	0.064	0.114
0.85	0.628	0.013	0.025

Best threshold by default-class F1: 0.55 (precision=0.339, recall=0.572)

```
plt.figure(figsize=(8,5))
plt.plot(threshold_table['threshold_on_p_default'], threshold_table['default_precision'], marker='o', label='Default Precision')
plt.plot(threshold_table['threshold_on_p_default'], threshold_table['default_recall'], marker='o', label='Default Recall')
plt.plot(threshold_table['threshold_on_p_default'], threshold_table['default_f1'], marker='o', label='Default F1')
plt.xlabel('Threshold on P(Charged Off)'); plt.ylabel('Score')
plt.title('Default-Class Precision / Recall / F1 vs Threshold')
plt.legend()
plt.show()
```

